

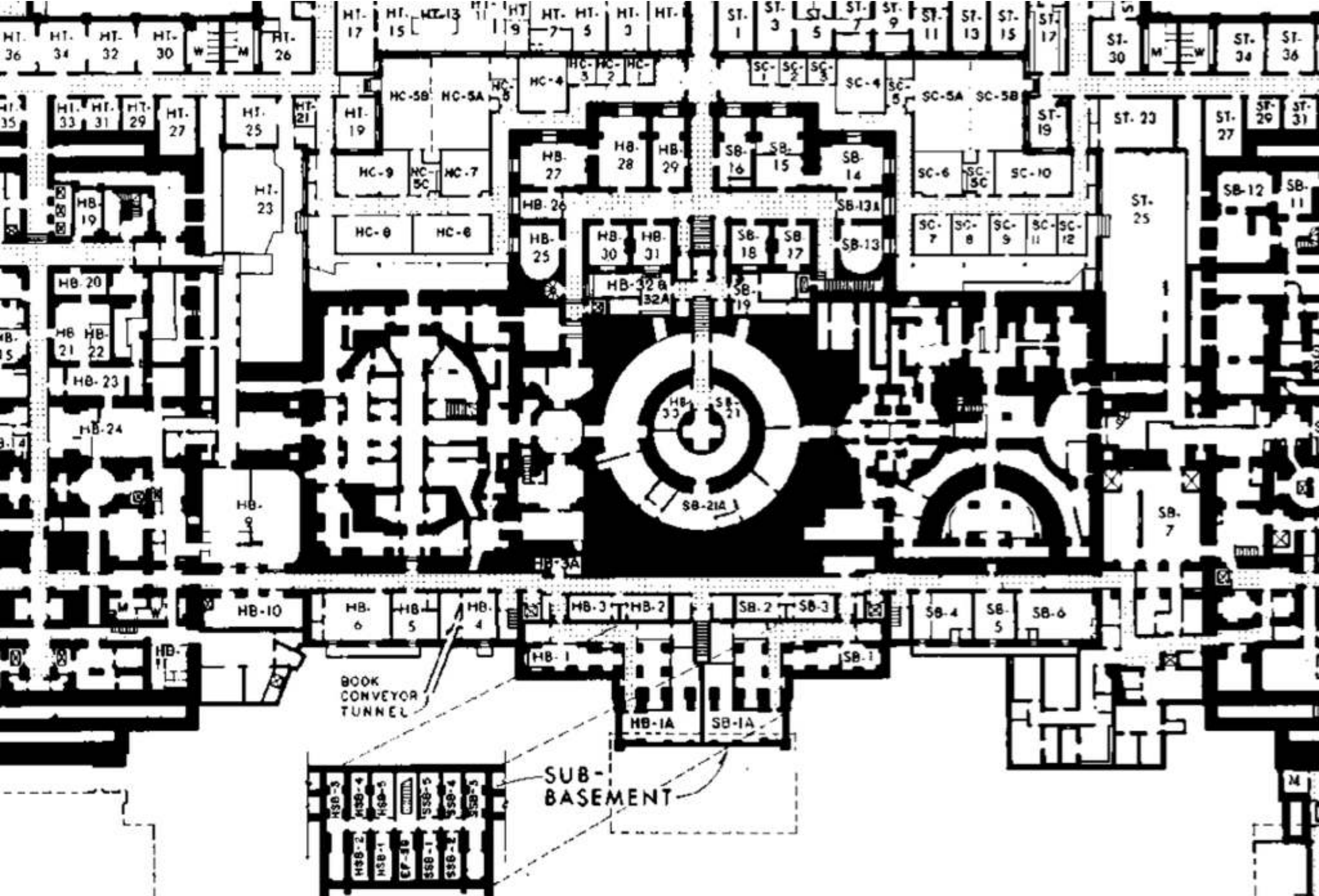
irrational → functional

**from irrational configuration system
to functional data store**

Rob Martin / @version2beta

OC Tanner Company

rob.martin@octanner.com ⇔ rob@version2beta.com



BASEMENT, TERRACE, AND COURTYARD
FLOOR PLAN

**"Things which grow shape themselves from within outwards –
they are not assemblages of originally distinct parts;
they partition themselves, elaborating their own structure
from the whole to the parts, from the simple to the complex."**

- Alan Watts

initial findings:

> 5250 configuration settings

42 tables

25% of settings have no effect

~20 systems mucking around directly in the database

~2,000,000 lines of Java

Few tests

*"It is easier to write an incorrect program
than understand a correct one."*

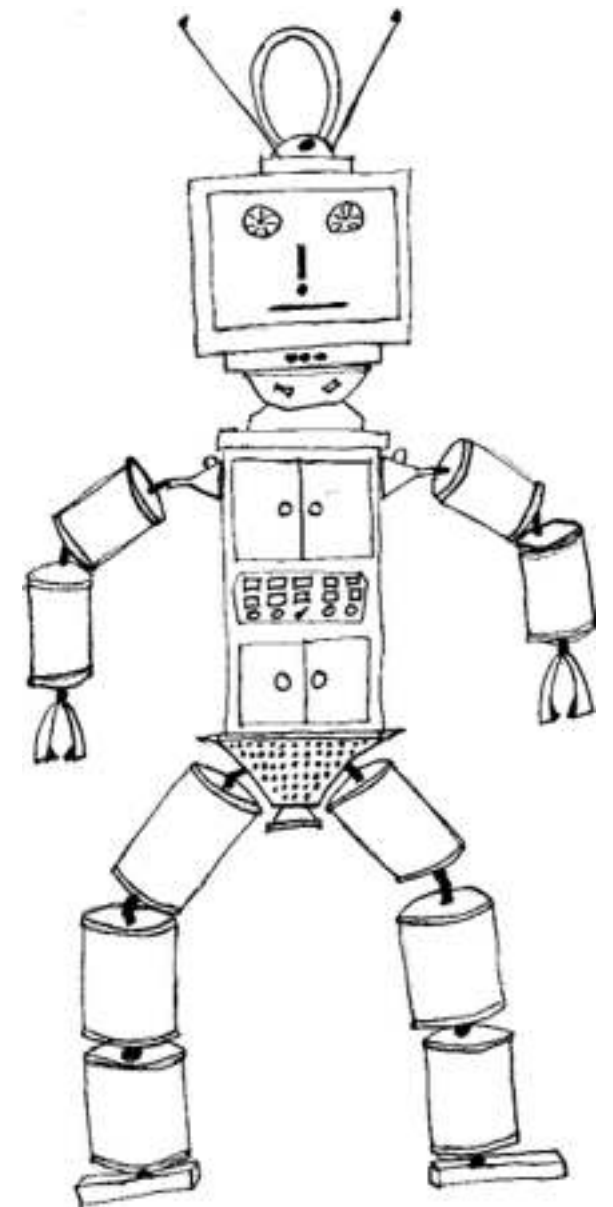
- Alan J. Perlis

design goals:

- Reduce complexity

how we did it:

- Design patterns
- Functional-first programming



"The price of reliability is the pursuit of the utmost simplicity." - C. A. R. Hoare

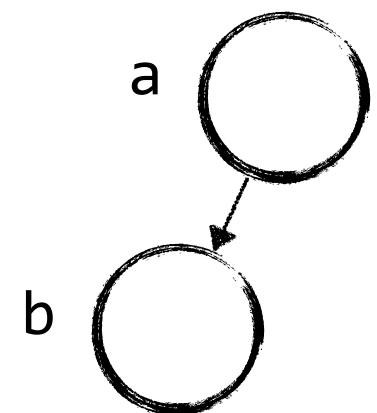
design patterns

Design patterns are abstract solutions for abstract problems.

1. Choose a problem.
2. Reduce it to an abstraction.
3. Find a pattern that fits the abstraction.
4. Choose a solution pattern that matches the problem pattern.
5. Implement the solution pattern to fit your specific problem.

“The purpose of software engineering is
to control complexity,
not to create it.”

- Dr Pamela Zave, AT&T Labs



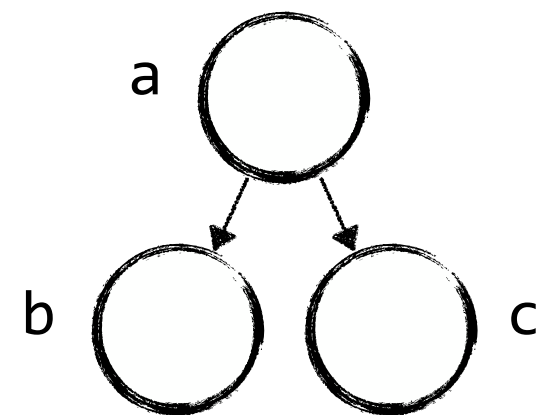
functional-first programming

1. Code everything that can be done without side-effect.
2. Code the side-effects.

(A mostly pure functional programming language is preferred, but not required.)

“If you’re willing to restrict the flexibility of your approach, you can almost always do something better.”

- John Carmack

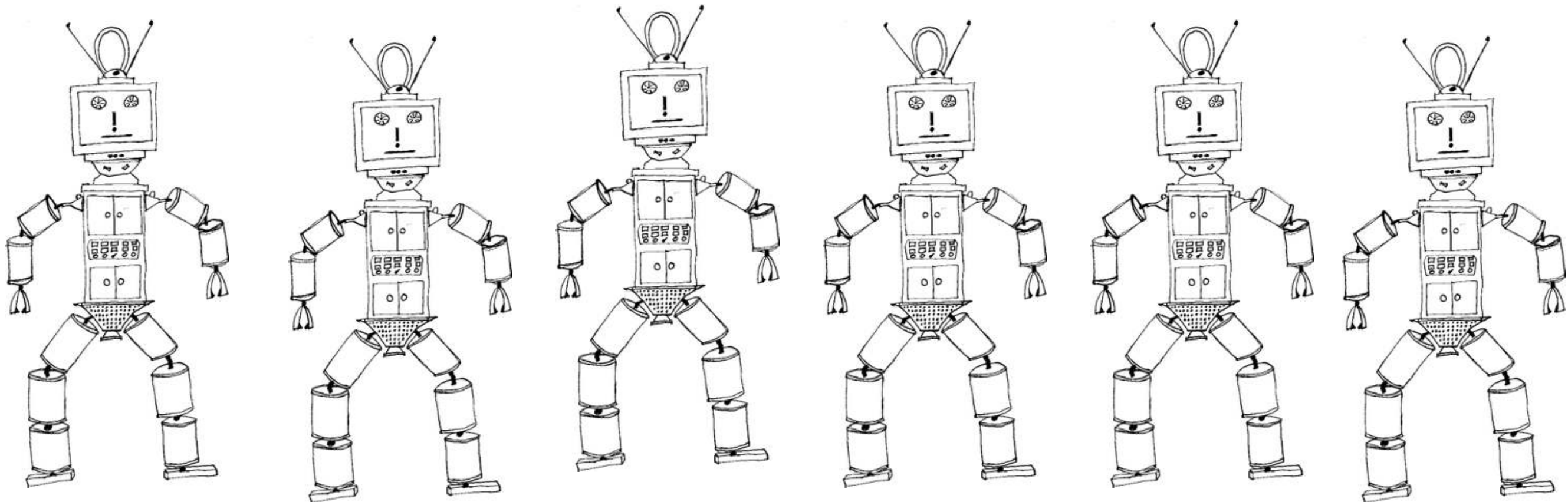


design goals:

- Multi-tenant
- Hierarchical key-value store with metadata.
- 10^7 keys, $<10\text{ms}$ query response time.

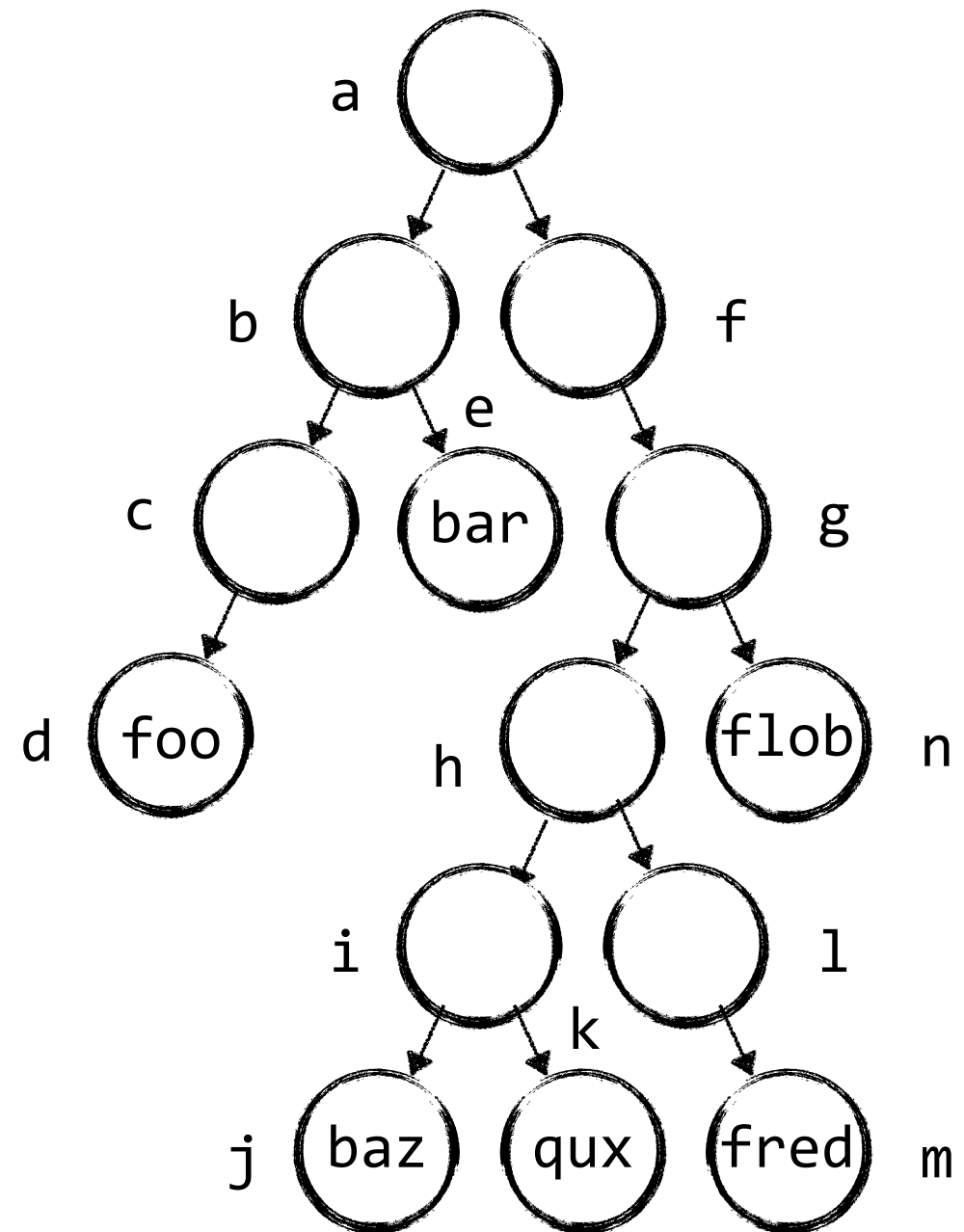
how we did it:

- Data structure



data structure (prefix trie)

a.b.c.d = "foo"
a.b.e = "bar"
a.f.g.h.i.j = "baz"
a.f.g.h.i.k = "qux"
a.f.g.h.l.m = "fred"
a.f.g.n = "flob"



design goals:

- A useful metaphor: books.
- Composable data.
- Time variant for version control.

how we did it:

- Immutable time variant prefix trie

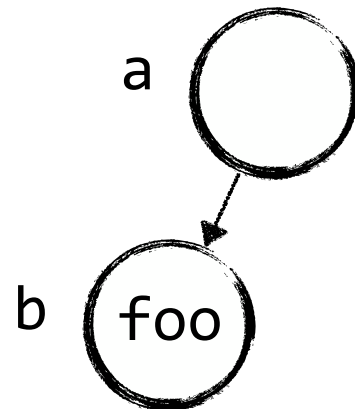
(This is an example of composable data.)

```
1 {
2   foo: {
3     bar: "baz"
4   }
5 }
6
7   {
8     foo: "bar",
9     bar: "baz"
10    boo: {
11      shoo: "yoo"
12    }
13  }
14
15   {
16     foo: {
17       bar: "baz"
18     },
19     bar: "baz",
20     boo: {
21       shoo: "yoo"
22     }
23   }
24 }
```

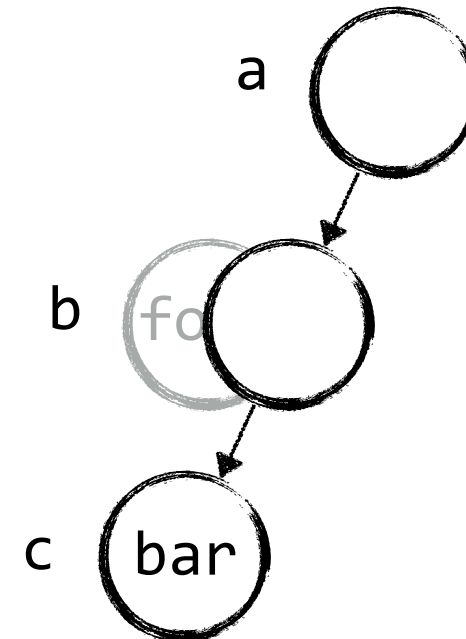
data structure

(immutable time variant prefix trie)

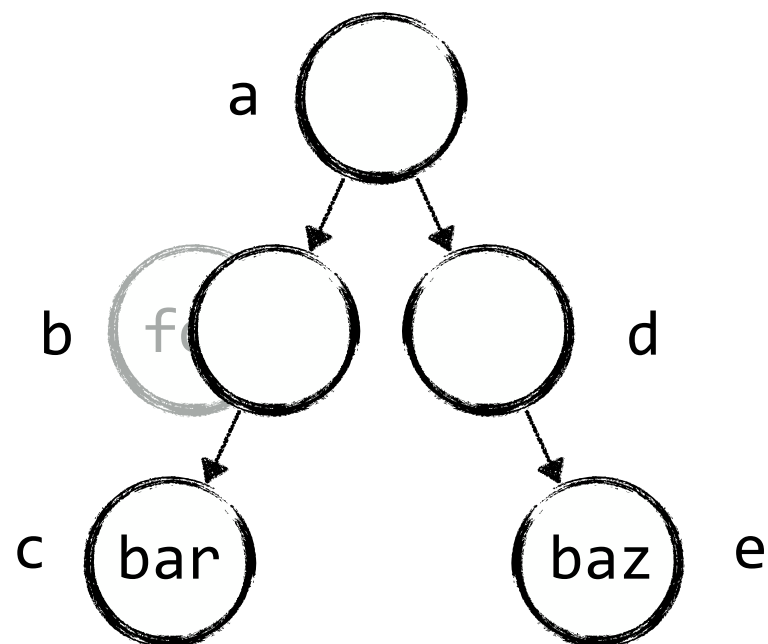
1> a.b = "foo"



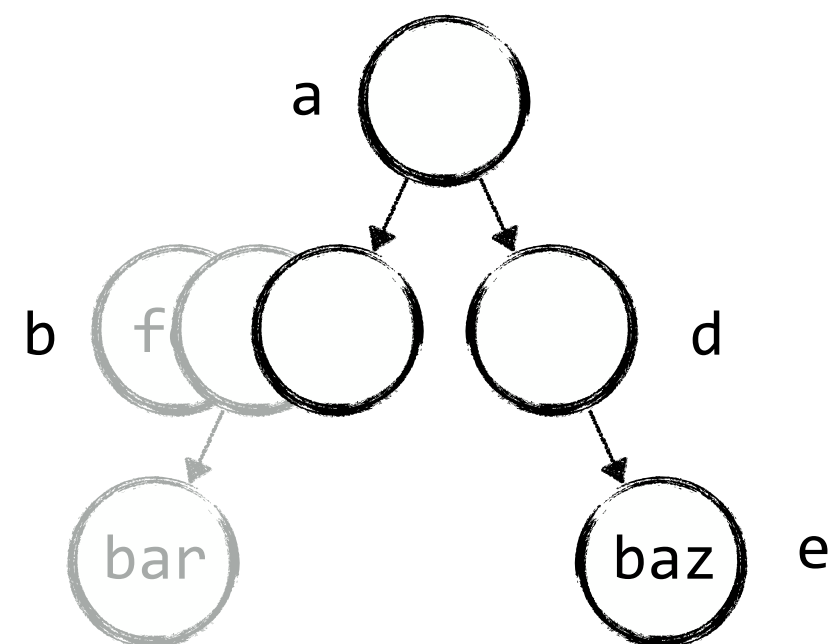
2> a.b.c = "bar"



3> a.d.e = "baz"



4> a.b.c = gone

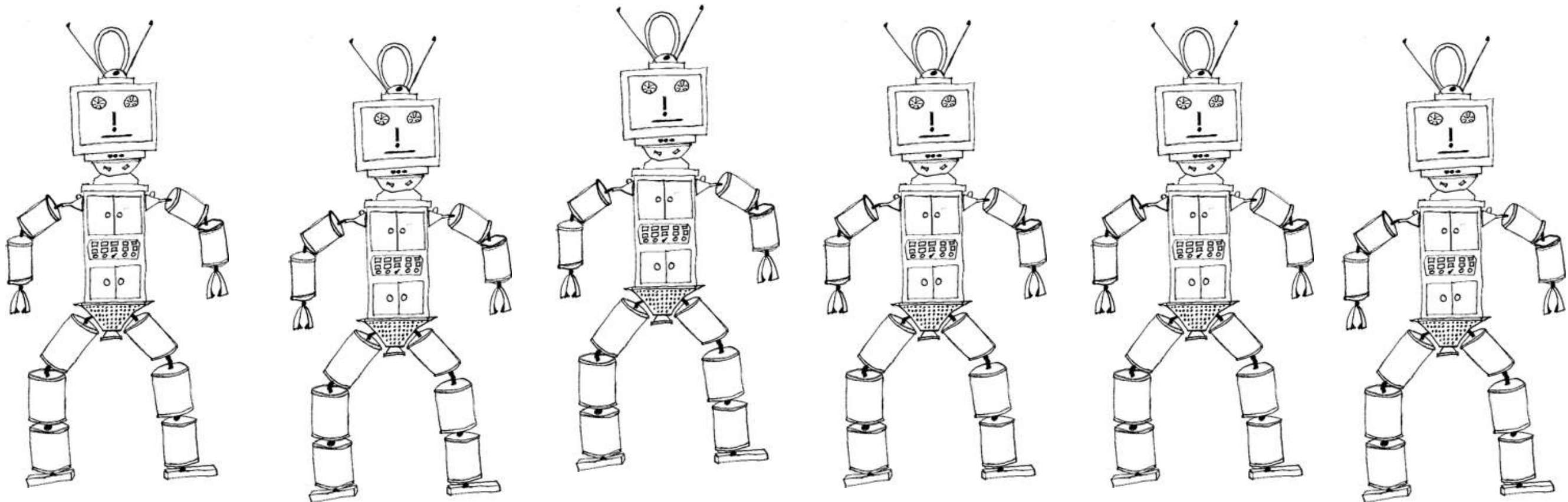


design goals:

- Single point of specification
- Documented and discoverable.
- Expose settings anywhere.

how we did it:

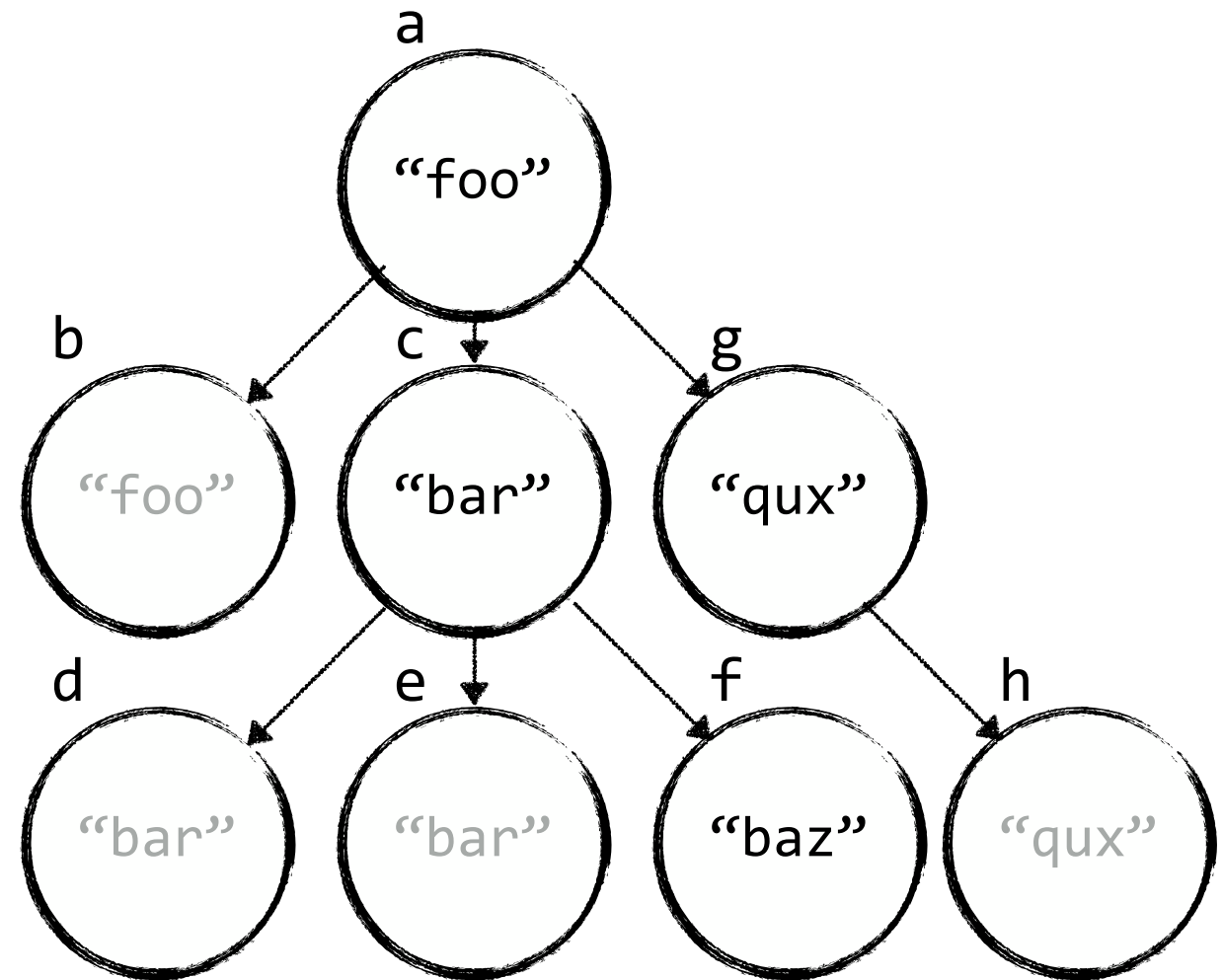
- Metadata



metadata

a._tag = "foo"
a.c._tag = "bar"
a.c.f._tag = "baz"
a.g._tag = "qux"

a._tag
"foo"
a.b._tag
"foo"
a.g.h._tag
"qux"

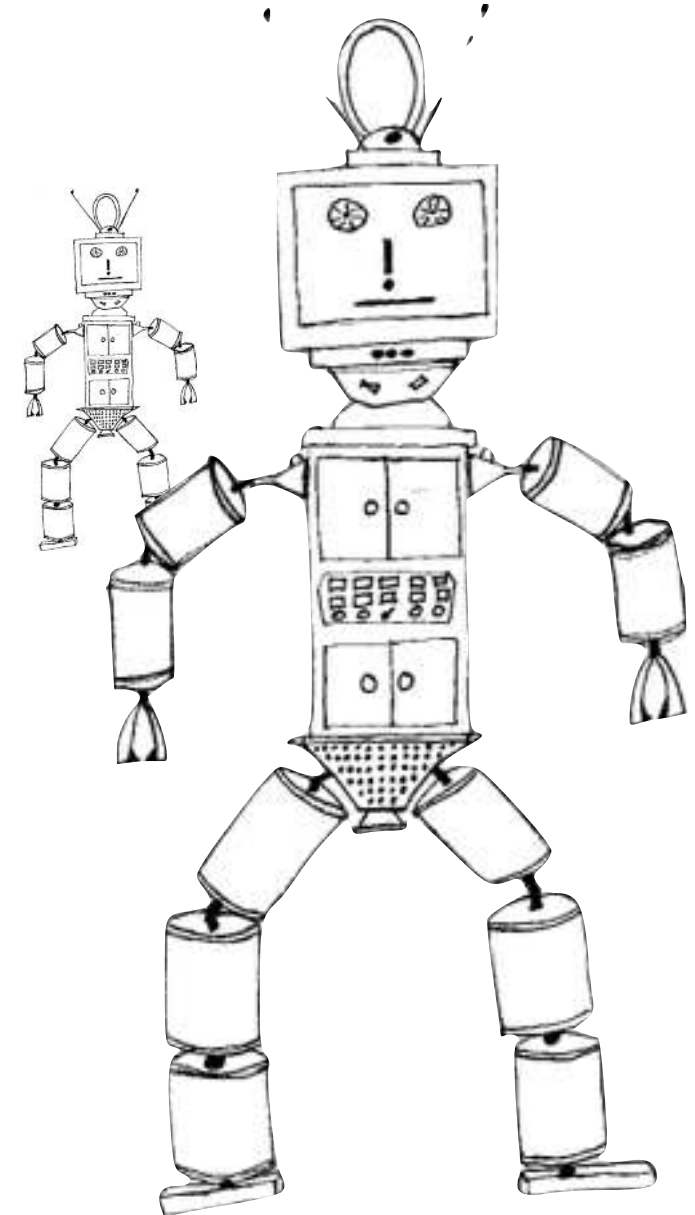


design goals:

- Don't require changes to legacy code.
- Foreign views.

how we did it:

- Functional data stores
- Data coordinator



functional data stores

- Event stores
- Command Query Responsibility Segregation (CQRS)

*"In the end, any program must manipulate state.
A program that has no side effects whatsoever is
a kind of black box. All you can tell is that the
box gets hotter."*

Simon Peyton-Jones

data coordinators

pub

sub

sub

pub

pub

sub

sub

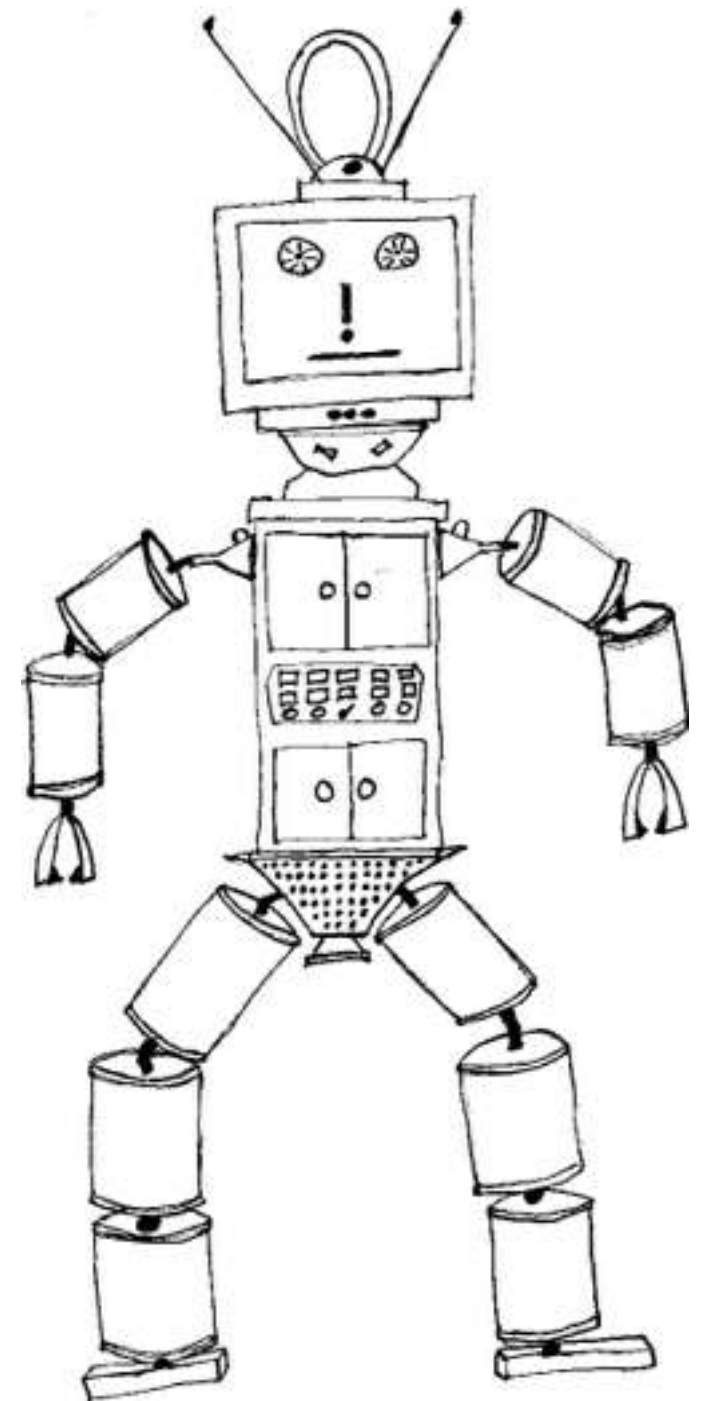
pub

pub

sub

sub

pub

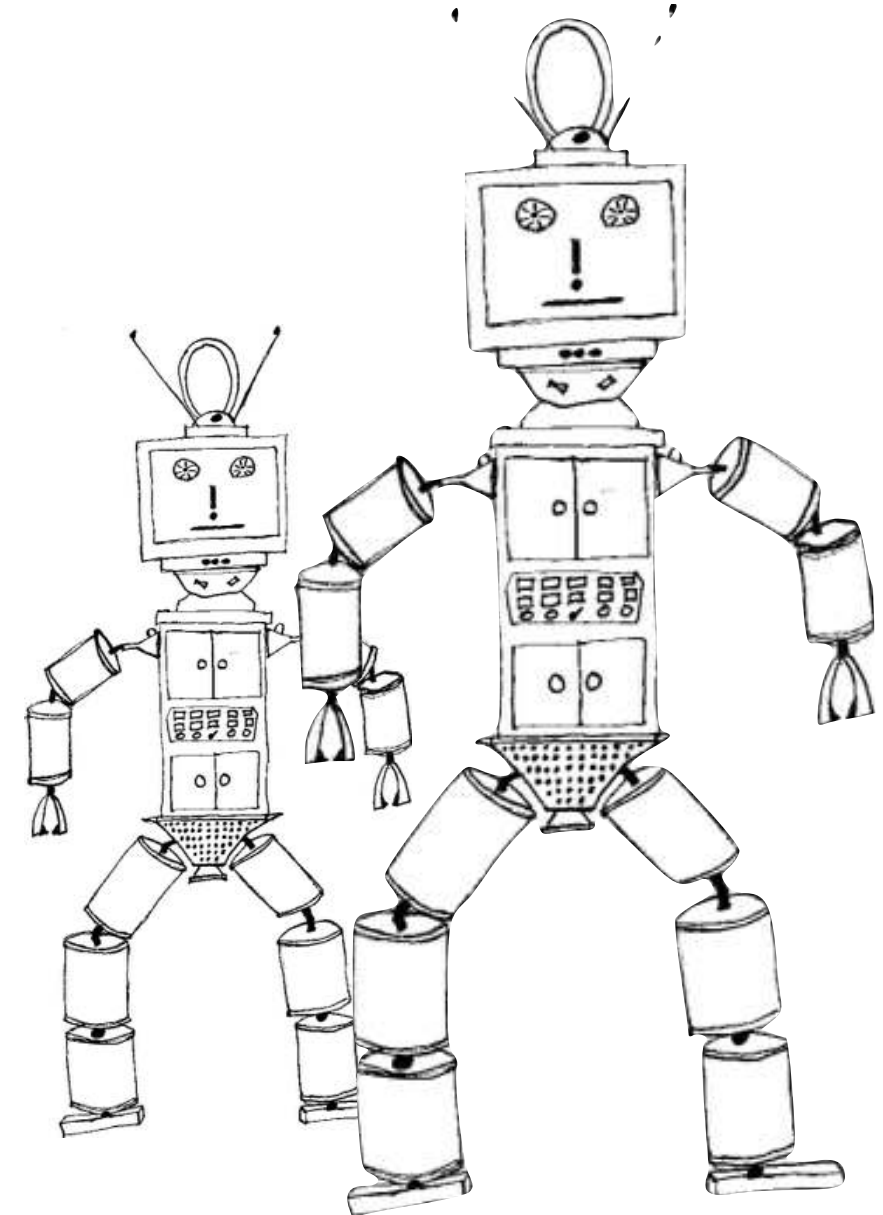


design goals:

- Build tools that support workflow, not the other way around.
- Expect internal customers to design their own tools.
- Deliver new tools the same day and iterate quickly.
- Make the tool building process lego-block simple. Juniors should do it.

how we did it:

- Separation of concerns, user-side and server-side
- Keep data sane
- config-tool-tool



separation of concerns

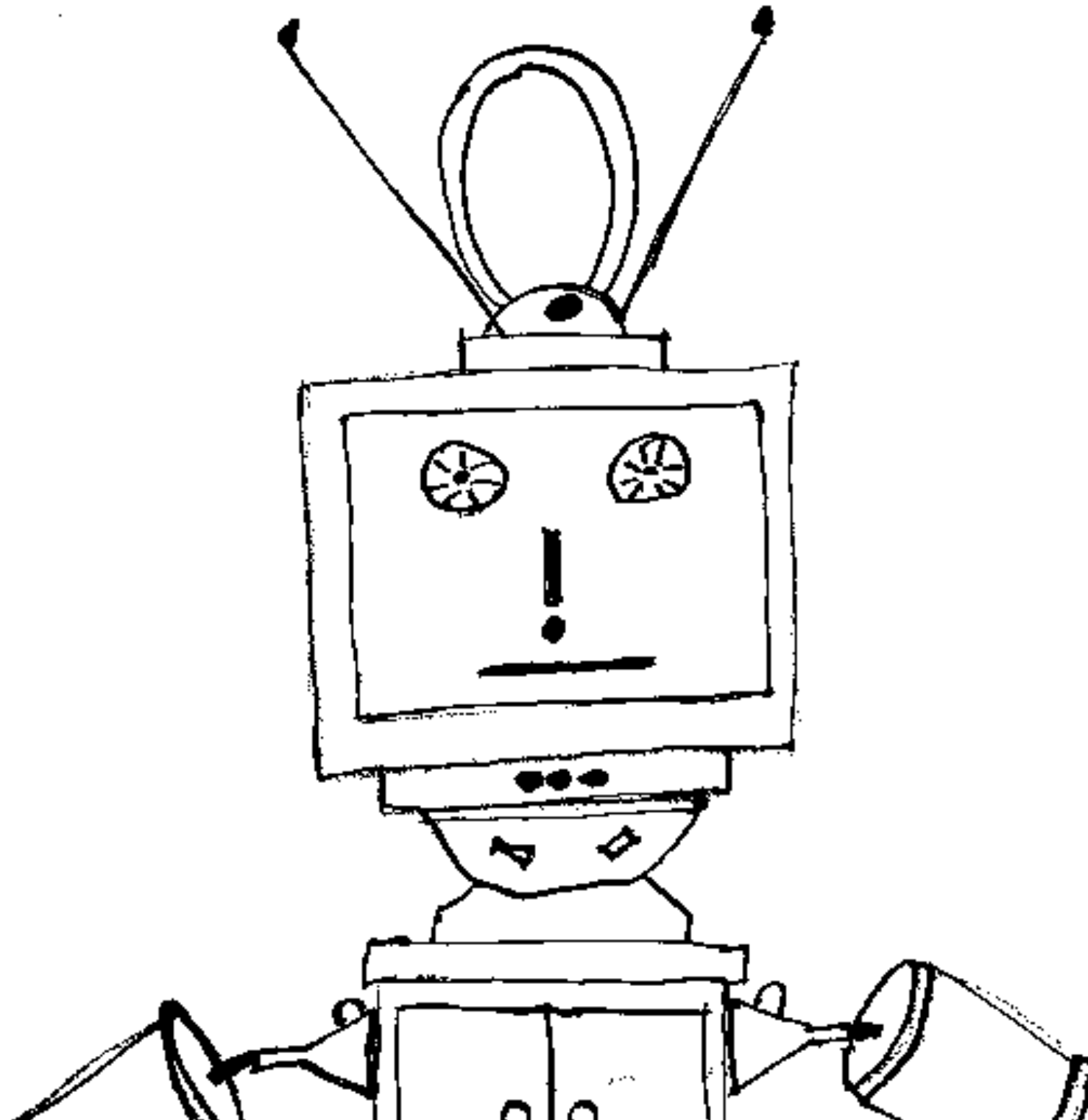
- Server-side code guarantees the integrity of data by applying business logic and using a competent persistence scheme.
- User-side code empowers the user to meet their needs and guarantees their positive experience.

"Simplicity does not precede complexity, but follows it."

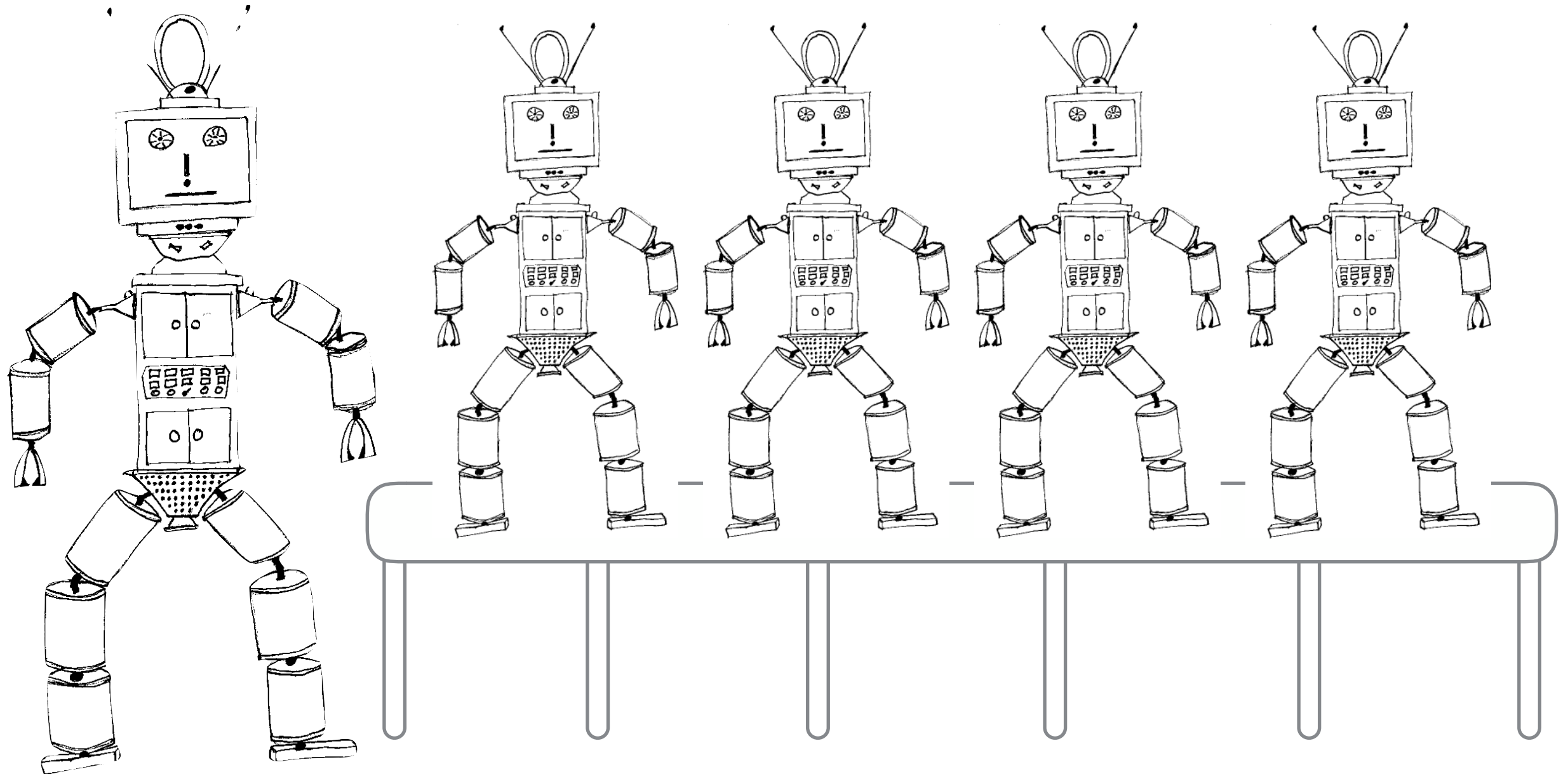
Alan J. Perlis

keep data sane

(the command side of command query responsibility segregation)



a config tool tool



a config tool tool build stack

Markdown

Custom
plugin

Template

```
---
title: Feature flags
---

Enable or disable features using the switches below.

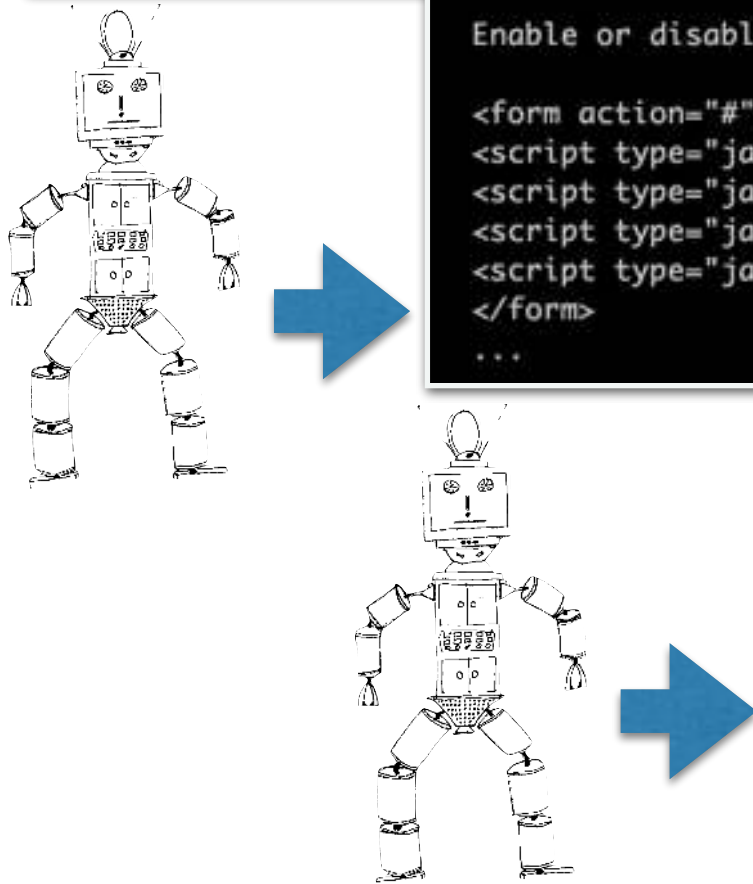
{{performance.fflags.newui}}
{{performance.fflags.wall}}
{{performance.fflags
{{performance.fflags
...
```

```
---
title: Feature flags
---

Enable or disable features using the switches below.

<form action="#">
<script type="javascript"> make_form_element("performance.fflags.newui"); </script>
<script type="javascript"> make_form_element("performance.fflags.wall"); </script>
<script type="javascript"> make_form_element("performance.fflags.profile.service"); </script>
<script type="javascript"> make_form_element("performance.fflags.profile.service"); </script>
</form>
...
```

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="stylesheet" href="..." />
    <script type="javascript" src="..." .../>
    ...
  </head>
  <body>
    <nav>
      ...
    </nav>
    <section>
      <h1>Feature flags</h1>
      <p>Enable or disable features using the switches below.</p>
      <form action="#">
        <script type="javascript"> make_form_element("performance.fflags.newui"); </script>
        ...
      </form>
    </section>
  </body>
</html>
```



Our first application wasn't configuration.

Nuestra primera aplicación no era configuración.

Vår första ansökan var inte konfiguration.

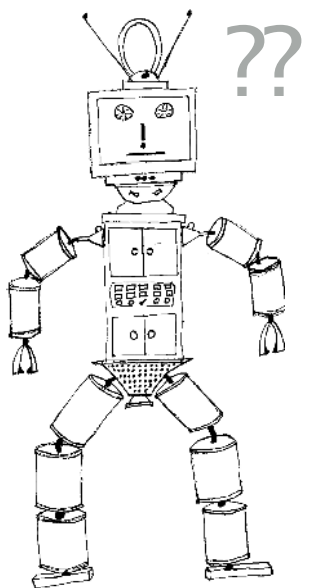
우리의 첫 번째 응용 프로그램이 구성되지 않았습니다.

Naša prva primjena nije bio konfiguracije.

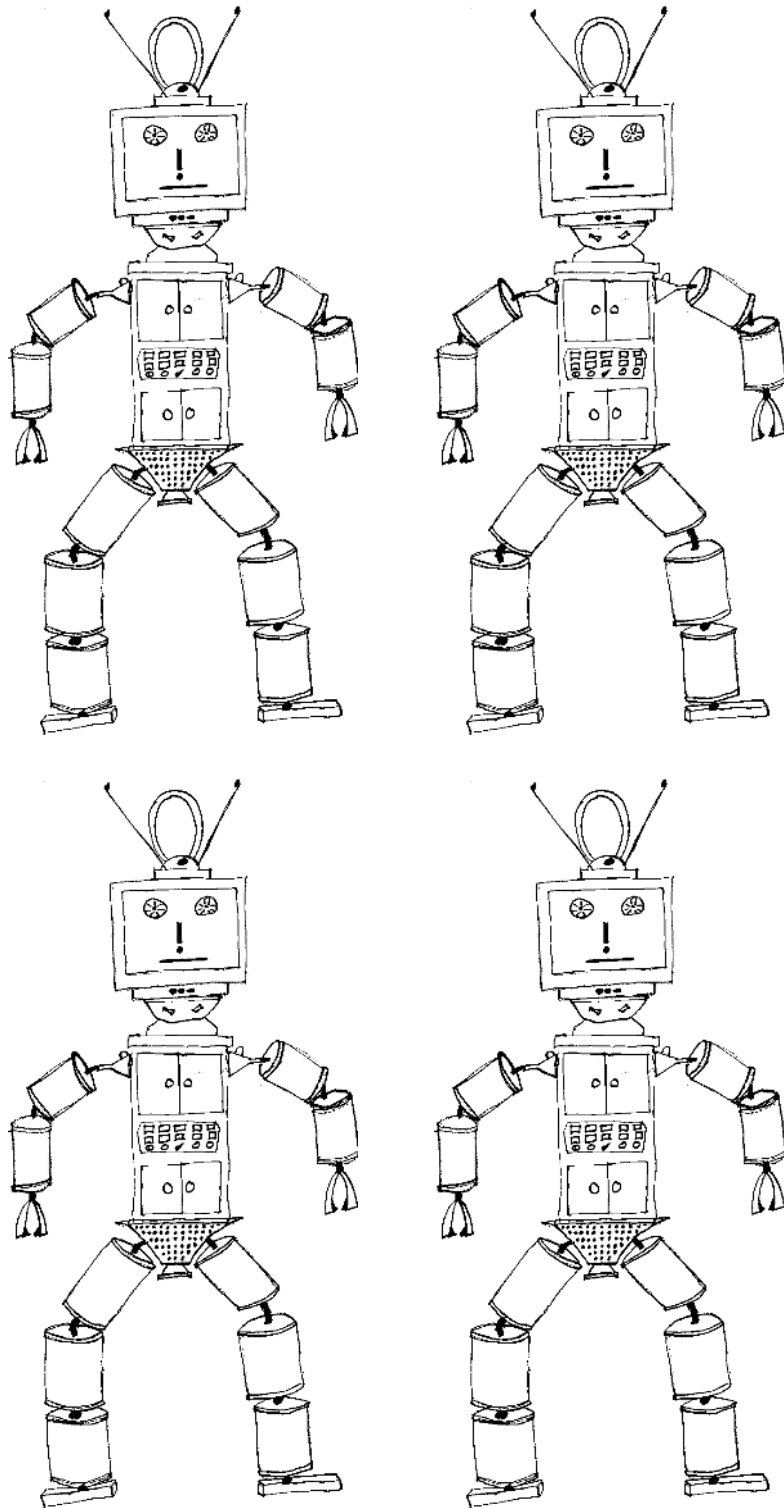
Onze eerste toepassing was niet configuratie.

Unsere erste Anwendung nicht Konfiguration.

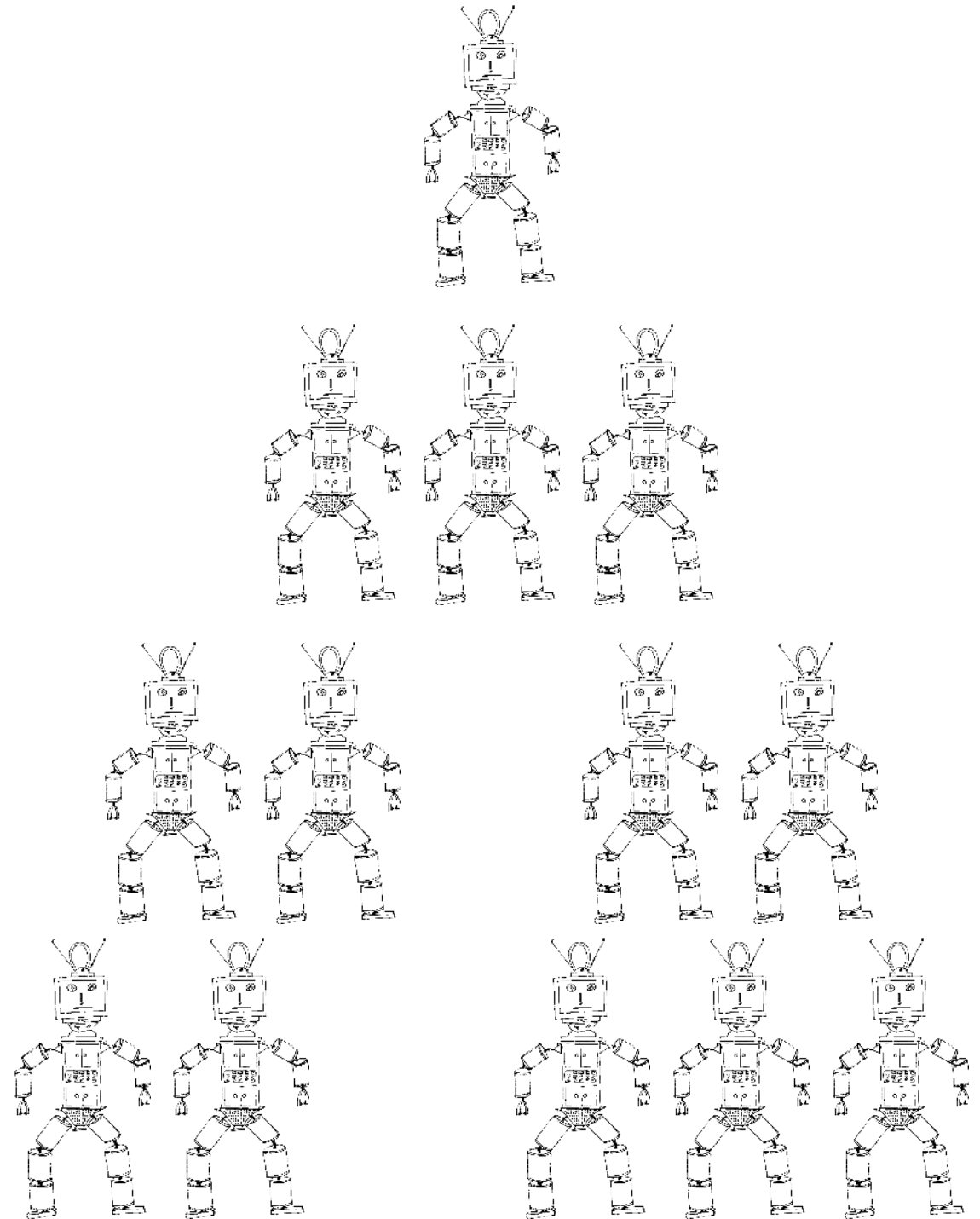
Nasza pierwsza aplikacja nie była System konfiguracja.



production java spike



development elixir version



what's next?

provably correct configurations!
(probably)

equality, which are easily explained using the fact that they hold, by definition, for canonical elements:

Reflexivity

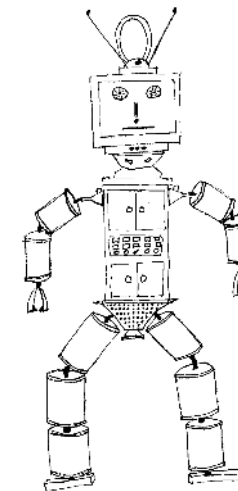
$$\frac{a \in A}{a = a \in A}$$

$$\frac{A \text{ set}}{A = A}$$

Symmetry

$$\frac{a = b \in A}{b = a \in A}$$

$$\frac{A = B}{B = A}$$



Transitivity

$$\frac{a = b \in A \quad b = c \in A}{a = c \in A}$$

$$\frac{A = B \quad B = C}{A = C}$$

For instance, a detailed explanation of transitivity is: $a = b \in A$ means that a and b yield canonical elements d and e , respectively, and that $d = e \in A$. Similarly if c yields f $e = f \in A$. Since we assume transitivity for canonical

"If debugging is the process of removing bugs,
then programming must be the process of putting them in."

- Edsger Dijkstra

version2beta.com/articles/irrational_to_functional

Rob Martin *from* OC Tanner Company *is* @version2beta
rob.martin@octanner.com ⇔ rob@version2beta.com

Appreciation to Michael Whitehead and our coworkers at OC Tanner Company,
and to my bosses for supporting this opportunity to share our work with you.